

Checklist de desarrollo por fases — Juego de carreras con Nakama

Oct 5, 2026 · @Jairo

Nueve fases, cada una con sus tareas de servidor, de cliente Unity y de pruebas como casillas para marcar. Una fase termina cuando todas las casillas de su criterio de terminado están marcadas. El diseño y el porqué de cada sistema están en [Doc](#).

Tablero

Fase	Estado	Depende de	Qué deja funcionando
1. Sesión de carrera y resultados	Sin empezar	Lo ya hecho	Un resultado oficial por carrera, validado por el servidor
2. Leaderboards y perfil	Sin empezar	Fase 1	Tablas reconstruidas que solo escribe el servidor
3. Economía, garaje y progresión	Sin empezar	Fase 1	Ganar monedas, comprar y mejorar autos, subir de nivel
4. Matchmaking y ranked	Sin empezar	Fases 1 y 3	Colas de 2, 4 y 6 autos, bots, rating y reconexión
5. Operación y lanzamiento en pruebas	Sin empezar	Fases 1 a 4	Servidor de producción, cuentas enlazadas, juego publicable
6. Misiones, logros y pase	Sin empezar	Fases 3 y 5	Misiones diarias, pase de temporada, cierre de temporada
7. Social	Sin empezar	Fase 5	Amigos, grupos, clubes y chat moderado
8. Torneos y eventos	Sin empezar	Fases 2, 4 y 5	Torneos con premio, calendario de eventos, telemetría
9. Compras reales y anuncios	Sin empezar	Fases 3 y 5	Venta de gemas, paquetes y pase en móvil y PC

En cada fase las casillas van en cuatro grupos: **Servidor** (una persona), **Cliente Unity** (una persona), **Pruebas** y **Criterio de terminado**. Servidor y cliente pueden avanzar en paralelo si acuerdan primero el contrato de cada RPC, que es siempre la primera casilla.

Fase 1 — Sesión de carrera y resultados

Objetivo: que cada carrera produzca un único resultado oficial en el servidor y un evento `RaceCompleted`. Todo el metajuego posterior se conecta a ese evento.

1.1 Preparación del proyecto de servidor

- ☐ Repositorio del servidor con proyecto TypeScript: `tsconfig`, tipos del runtime de Nakama y empaquetado a un solo `index.js`
- ☐ Docker Compose local con Nakama y PostgreSQL, y el módulo compilado montado en el contenedor
- ☐ Estructura de carpetas: un directorio por módulo y un `main.ts` con `InitModule` que registra todo
- ☐ README con la convención de nombres de RPC, el formato de respuesta `{ok, data, error}` y la lista de códigos de error
- ☐ Linter y pruebas unitarias de lógica pura (sin Nakama) en un solo comando
- ☐ Documento con los códigos de operación del protocolo de carrera actual

1.2 Contratos

- ☐ Contrato escrito (entrada, salida, errores) de `config_get`, `race_session_create`, `race_session_join`, `race_session_start`, `race_session_get` y `race_submit_result`
- ☐ Forma del evento `RaceCompleted` acordada: sesión, modo, pista, tamaño, lista ordenada de participantes con tiempos, confianza y marcas

1.3 Servidor: módulo `core`

- ☐ Carga de catálogos JSON al arrancar, con validación de esquema; un catálogo inválido impide el arranque
- ☐ Catálogo `tracks`: id, vueltas por modo, número de checkpoints y tiempo mínimo plausible por clase de auto
- ☐ Catálogo `modes`: rápida, ranked, privada y contrarreloj, con tamaños permitidos (2, 4, 6) y multiplicadores
- ☐ `config_get`: catálogos públicos, hash, hora del servidor, versión mínima de cliente

- ☐ Utilidad de idempotencia: clave registrada devuelve el resultado guardado en la segunda llamada
- ☐ Utilidad de errores tipados y de log que incluye siempre el ID de sesión de carrera
- ☐ Utilidad de límite de frecuencia por jugador y por RPC
- ☐ Bus interno de eventos: los módulos se suscriben a `RaceCompleted` en `InitModule`; si un suscriptor falla, se registra y los demás continúan
- ☐ Campo `schemaVersion` y función de migración al leer en el ayudante de storage

1.4 Servidor: módulo `race`

- ☐ Modelo `RaceSession`: id, ID de match, modo, pista, tamaño, roster (jugador, loadout, es bot), host, sucesión de host, estado, `startedAt`, resultados
- ☐ Estados de sesión: `created`, `started`, `closing`, `closed`, con transiciones válidas comprobadas
- ☐ `race_session_create` para salas privadas: el host envía el ID del match creado desde el cliente, modo, pista y tamaño
- ☐ `race_session_join`: cada jugador de la sala confirma su entrada; el servidor lo añade al roster mientras haya cupo
- ☐ Índice de sesión activa por jugador, para saber si tiene una carrera viva
- ☐ `race_session_start`: solo el host; sella `startedAt` con el reloj del servidor y devuelve la hora de salida
- ☐ `race_session_get`: devuelve la sesión viva del jugador o la última cerrada
- ☐ `race_submit_result`: validación de roster, estado de sesión y reporte no duplicado
- ☐ Validación de reloj: el tiempo reportado no supera el transcurrido desde `startedAt`
- ☐ Validación de tiempo mínimo plausible de la pista para la clase del auto
- ☐ Validación de vueltas: cantidad correcta y suma igual al total
- ☐ Reporte de bots aceptado solo desde el host
- ☐ Cierre de sesión: cuando todos reportaron o 30 s después del primer reporte; se evalúa en cada llamada a la sesión
- ☐ Quien no reportó al cierre queda como abandono
- ☐ Cálculo del orden oficial por tiempo total, con abandonos al final
- ☐ Quórum: comparar el orden visto por cada cliente y asignar confianza `quorum` o `client`
- ☐ Registro de marcas de revisión con el motivo
- ☐ Emisión de `RaceCompleted` exactamente una vez por sesión, protegida con escritura condicional por versión

- ☐ Respuesta de `race_submit_result` y de `race_session_get` con el resultado oficial cuando la sesión está cerrada

1.5 Cliente Unity

- ☐ Interfaces `IConfigService`, `IRaceSessionService` e `IServerClock` con implementación Nakama, registradas en el contenedor
- ☐ Reloj de servidor: diferencia calculada con varias muestras y mediana
- ☐ Descarga de catálogos y caché local por hash
- ☐ Sala privada: tras crear el match, el host llama `race_session_create` y los demás `race_session_join`
- ☐ Pista, vueltas y loadouts de todos los autos aplicados desde la sesión
- ☐ Cuenta regresiva basada en la hora de salida del servidor
- ☐ Mensajes de checkpoint y de meta con código de operación propio, separados del estado del auto
- ☐ Registro local de tiempo por vuelta y parcial por checkpoint
- ☐ Envío del resultado al cruzar la meta, con reintentos y cola si no hay red
- ☐ El host reporta los resultados de los bots
- ☐ Pantalla de resultados pintada con la respuesta del servidor; consulta la sesión cada 2 s hasta el cierre
- ☐ Mensajes para resultado rechazado y sesión no encontrada

1.6 Pruebas

- ☐ Pruebas unitarias de orden oficial, quórum y cada validación
- ☐ Carrera completa con 2, 4 y 6 clientes reales
- ☐ Reenviar el mismo resultado devuelve la misma respuesta y no emite otro evento
- ☐ Tiempo menor al mínimo plausible: rechazado
- ☐ Tiempo mayor al transcurrido según el servidor: rechazado
- ☐ Jugador fuera del roster: rechazado
- ☐ Cliente que no reporta: abandono al vencer el plazo
- ☐ Dos clientes con órdenes distintos: confianza `client` y marca de revisión
- ☐ Corte de red al cruzar la meta: el resultado llega al recuperar la conexión

1.7 Criterio de terminado

- ☐ Una carrera de 4 clientes reales produce un único resultado oficial con orden y confianza
- ☐ Un suscriptor de prueba recibe `RaceCompleted` una sola vez por carrera

- ☐ Ningún resultado imposible queda aceptado
- ☐ El protocolo de carrera ya envía checkpoint y meta como mensajes propios

Fase 2 — Leaderboards y perfil

Objetivo: reconstruir las leaderboards desde cero para que solo las escriba el servidor a partir de `RaceCompleted`, y dar a cada jugador un perfil público. La leaderboard actual escrita por el cliente se descarta.

2.1 Contratos

- ☐ Contrato escrito de `lb_get` (tabla, vista, cursor) y de `profile_get` / `profile_update`
- ☐ Forma única de fila de leaderboard: puesto, jugador, nombre, avatar, división, club, puntaje, metadata

2.2 Servidor: módulo `leaderboards`

- ☐ Catálogo `leaderboards` con el registro de tablas: patrón de ID, operador, orden, reinicio
- ☐ Creación idempotente de todas las tablas al arrancar, como autoritativas
- ☐ Tablas de tiempo por pista y clase: `tt_{pista}_{clase}_all` y `tt_{pista}_{clase}_week`
- ☐ Tablas de mejor vuelta: `lap_{pista}_{clase}_all`
- ☐ Tabla `wins_week` de victorias semanales
- ☐ Suscripción a `RaceCompleted`: escribe tiempos, mejor vuelta y victorias según modo y confianza
- ☐ Regla de confianza: `quorum` y `server` entran siempre; `client` solo en contrarreloj validado y marcado en metadata
- ☐ Metadata por registro: auto, plataforma, control, versión de cliente, ID de sesión
- ☐ Tiempos guardados en milisegundos; subpuntaje con marca de tiempo para desempate
- ☐ `lb_get` vista global con paginación por cursor
- ☐ `lb_get` vista alrededor del jugador
- ☐ `lb_get` vista de amigos (lista de amigos más el jugador)
- ☐ Enriquecimiento de filas con datos de perfil en una sola lectura por lote
- ☐ Función `removePlayerFromAll(userId)` para retirar a un jugador de todas las tablas
- ☐ Hook `before` que rechaza cualquier escritura de leaderboard hecha desde un cliente
- ☐ Borrado de la tabla antigua escrita por el cliente

2.3 Servidor: módulo `progression` (perfil)

- ☐ Objeto `profile/main` con nombre visible, avatar, nivel, XP, auto mostrado, plataforma
- ☐ Creación del perfil en el primer inicio de sesión (hook posterior a la autenticación)
- ☐ `profile_get` propio y de otro jugador (solo campos públicos)
- ☐ `profile_update` para nombre y avatar, con validación de longitud y caracteres
- ☐ Lista básica de palabras bloqueadas aplicada al nombre

2.4 Cliente Unity

- ☐ Interfaces `ILeaderboardService` e `IProfileService` con implementación Nakama
- ☐ Eliminado todo envío de puntajes desde el cliente
- ☐ Pantalla de leaderboards: selector de pista, clase y periodo (semana, histórico)
- ☐ Pestañas global, alrededor de mí y amigos, con paginación
- ☐ Fila del jugador siempre visible y resaltada
- ☐ Formato de tiempo mm:ss.mmm a partir de milisegundos
- ☐ Pantalla de perfil propio con edición de nombre y avatar
- ☐ Ficha de perfil de otro jugador desde cualquier fila
- ☐ Aviso de mejor marca personal en la pantalla de resultados
- ☐ Estados de carga, vacío y error en cada vista

2.5 Pruebas

- ☐ Una carrera válida actualiza tiempo por pista, mejor vuelta y victorias
- ☐ Un tiempo peor que el registrado no reemplaza la marca
- ☐ Un resultado con confianza `client` en carrera con rivales no entra a las tablas de tiempo
- ☐ Escritura directa desde un cliente: rechazada por el hook
- ☐ Reinicio semanal simulado en local: la tabla semanal queda vacía y la histórica intacta
- ☐ Vista alrededor de mí con el jugador en el primer puesto, en el último y sin registro
- ☐ Nombre con palabra bloqueada: rechazado

2.6 Criterio de terminado

- ☐ Terminar una carrera actualiza las tablas sin que el cliente envíe puntajes
- ☐ Las tres vistas responden con filas enriquecidas en una sola llamada
- ☐ No queda ninguna tabla que un cliente pueda escribir
- ☐ Cada jugador tiene perfil y puede ver el de los demás

Fase 3 — Economía, garaje y progresión

Objetivo: que correr dé monedas y XP, y que eso se convierta en autos, mejoras y cosméticos que los rivales ven aplicados en la carrera.

3.1 Diseño y contratos

- ☐ Hoja de cálculo de economía: monedas por hora de juego, costo de cada auto y mejora, días hasta el primer auto nuevo (objetivo: 2 a 3)
- ☐ Lista de los 6 autos iniciales (o más) con clase y estadísticas base y tope
- ☐ Lista inicial de cosméticos por tipo: pintura, llantas, calcomanía, estela, bocina
- ☐ Tabla de XP por nivel (1 a 50) y desbloques por nivel
- ☐ Contrato escrito de `garage_get`, `car_buy`, `car_upgrade`, `cosmetic_equip`, `loadout_set`, `store_get`, `store_buy`, `wallet_get`

3.2 Servidor: módulo `economy`

- ☐ Monedas `coins` y `gems` en el wallet de Nakama
- ☐ Función única `grant(userId, recompensa, motivo, claveIdempotencia)` usada por todos los módulos
- ☐ Función única `spend(userId, costo, motivo)` que falla si el saldo no alcanza
- ☐ Motivo e ID de origen en la metadata de cada movimiento del ledger
- ☐ Tipo `Reward` común: monedas, gemas, XP, auto, cosmético; un solo punto que sabe entregar cada tipo
- ☐ Catálogo `rewards`: base por posición para 2, 4 y 6 autos, multiplicadores por modo, bonos
- ☐ Suscripción a `RaceCompleted`: paga monedas y XP por posición, con el ID de sesión como clave de idempotencia
- ☐ Bono de primera victoria del día con sello de fecha UTC
- ☐ Premio reducido en sala privada y tope diario de carreras privadas premiadas
- ☐ Catálogo `store`: secciones, ofertas, precios, requisitos de nivel, vigencia
- ☐ Oferta diaria rotativa con semilla por fecha
- ☐ `store_get` filtrado por nivel, propiedad y eventos activos
- ☐ `store_buy` con verificación de vigencia y requisitos, cobro y entrega en una transacción
- ☐ `wallet_get` con saldos

3.3 Servidor: módulo `garage`

- ☐ Catálogo `cars` : id, clase, estadísticas base, estadísticas tope, precio, nivel requerido
- ☐ Catálogo `upgrades` : 4 líneas, 5 niveles, costo y efecto por nivel
- ☐ Catálogo `cosmetics` : id, tipo, rareza, precio, autos compatibles
- ☐ Objeto `garage/cars` con autos poseídos, niveles de mejora y cosméticos
- ☐ Auto inicial entregado al crear la cuenta
- ☐ `garage_get` con el garaje completo y las estadísticas calculadas de cada auto
- ☐ `car_buy` : nivel requerido, no poseído, cobro y entrega en una sola transacción (`multiUpdate`)
- ☐ `car_upgrade` : siguiente nivel de una línea, sin superar el tope de clase
- ☐ `cosmetic_equip` con verificación de propiedad y compatibilidad
- ☐ `loadout_set` : valida propiedad y escribe `garage/loadout` público
- ☐ Función `computeStats(auto, mejoras)` compartida, y variante igualada al tope de clase para ranked
- ☐ La sesión de carrera copia el loadout y las estadísticas de cada jugador al fijarse

3.4 Servidor: módulo `progression`

- ☐ Suscripción a `RaceCompleted` : suma XP
- ☐ Subida de nivel, incluso varios niveles de una vez, con su premio por nivel
- ☐ Desbloques por nivel leídos del catálogo y expuestos en `profile_get`
- ☐ La respuesta de resultados incluye monedas, XP, niveles subidos y desbloques nuevos

3.5 Cliente Unity

- ☐ Interfaces `IEconomyService`, `IGarageService`, `IStoreService` con implementación Nakama
- ☐ Saldo visible en el menú, actualizados tras cada RPC que los cambia
- ☐ Pantalla de garaje: lista de autos, estadísticas en barras, estado bloqueado, comprable o poseído
- ☐ Compra de auto con confirmación y mensaje de saldo insuficiente
- ☐ Pantalla de mejoras: 4 líneas, costo del siguiente nivel, vista previa del cambio de estadísticas
- ☐ Pantalla de personalización: cosméticos por tipo con vista previa en el auto
- ☐ Selección de auto activo antes de entrar a una carrera

- ☐ Autos rivales contruidos con el loadout y las estadísticas de la sesión, no con datos enviados por el rival
- ☐ Tienda: secciones, oferta diaria con cuenta regresiva, compra con confirmación
- ☐ Pantalla de resultados con monedas, XP, barra de nivel y desbloques
- ☐ Pantalla de subida de nivel
- ☐ El cliente nunca calcula precios ni premios: solo muestra lo que devuelve el servidor

3.6 Pruebas

- ☐ Pruebas unitarias de `computeStats`, premios por posición y subida de nivel
- ☐ Comprar sin saldo: rechazado, sin cambios
- ☐ Comprar un auto ya poseído: rechazado
- ☐ Mejorar por encima del nivel 5: rechazado
- ☐ Equipar un cosmético no poseído: rechazado
- ☐ Fallo forzado entre cobro y entrega: no queda cobro sin entrega
- ☐ El mismo `RaceCompleted` procesado dos veces paga una sola vez
- ☐ Cambiar el precio en el catálogo se refleja en el cliente sin build
- ☐ Jugador nuevo simulado: llega al primer auto nuevo en el plazo objetivo

3.7 Criterio de terminado

- ☐ Un jugador nuevo gana monedas corriendo, compra un auto, lo mejora y lo personaliza
- ☐ Los rivales ven ese auto con sus estadísticas y cosméticos leídos de la sesión
- ☐ Cada movimiento del wallet aparece en el ledger con su motivo
- ☐ Ningún precio ni premio está escrito en el código del cliente

Fase 4 — Matchmaking y ranked

Objetivo: que un jugador entre a una cola y termine en una carrera de 2, 4 o 6 autos con rivales de su nivel, con bots cuando falte gente en carrera rápida, y que el ranked mueva un rating confiable.

4.1 Diseño y contratos

- ☐ Propiedades y consulta del ticket escritas para cada modo: `mode`, `version`, `region`, `size`, `rating`, `class`, `platform`, `input`, `rtt`
- ☐ Regla de tamaños: en rápida el jugador elige 2, 4 o 6 y los bots completan; en ranked hay una sola cola sin bots que forma carreras de 2, 4 o 6 humanos (mínimo 2, máximo 6, múltiplos de 2)

- ☐ Contrato escrito de `ranked_get`, `mm_ticket_params` y `race_session_quick_bots`
- ☐ Parámetros de rating en el catálogo `ranked`: K inicial y normal, rating inicial, límites de división

4.2 Servidor: módulo `matchmaking`

- ☐ `mm_ticket_params`: devuelve al cliente las propiedades y la consulta que debe usar, según modo y configuración
- ☐ Hook `matchmakerMatched`: valida que todos los tickets coincidan en modo, versión y región
- ☐ El hook crea la `RaceSession` con el roster emparejado y deja el match como relay
- ☐ Elección de pista: aleatoria entre las desbloqueadas por todos, sin repetir la última de cada jugador
- ☐ Elección de host por menor `rtt` y lista de sucesión ordenada
- ☐ Relleno con bots: la sesión indica cuántos bots y de qué dificultad según el rating medio
- ☐ `race_session_quick_bots`: crea una sesión de carrera rápida de un humano con bots cuando la cola vence sin rivales
- ☐ Soporte de Parties: el grupo entra con un solo ticket y su rating es el más alto de sus miembros
- ☐ Clave `mm.segmentBy` en `liveops/config` con valores `none`, `input`, `platform`
- ☐ Tiempos de espera y ventanas de rating en configuración remota
- ☐ Evento de analítica por emparejamiento: tiempo en cola, humanos, bots, diferencia de rating

4.3 Servidor: reconexión y abandono

- ☐ `race_session_get` devuelve ID de match y estado para reincorporarse
- ☐ Marca de desconexión por jugador reportada por el host, con hora del servidor
- ☐ Gracia de 20 s; pasado ese tiempo el jugador queda como abandono
- ☐ RPC `race_host_claim`: el siguiente en la sucesión toma el rol de host; el servidor lo confirma una sola vez
- ☐ Contador de abandonos en ranked por 24 h y bloqueo de cola de 15 minutos al tercero

4.4 Servidor: módulo `ranked`

- ☐ Objeto `ranked/s{N}` por jugador: rating, división, división más alta, carreras jugadas
- ☐ Catálogo `seasons` con la temporada 1 y sus fechas
- ☐ Función de rating para N humanos según la fórmula del plan, con pruebas unitarias

- ☐ Suscripción a `RaceCompleted` en modo ranked: calcula y guarda el cambio de rating de cada humano
- ☐ Solo resultados con confianza `quorum` o `server` mueven rating
- ☐ El abandono cuenta como último lugar
- ☐ Tabla `ranked_s1` actualizada con operador `set`
- ☐ Estadísticas igualadas al tope de clase en las sesiones ranked
- ☐ Ranked bloqueado hasta el nivel 5
- ☐ `ranked_get` : rating, división, progreso a la siguiente, puesto en la tabla, días restantes de temporada
- ☐ La respuesta de resultados incluye rating anterior, nuevo y cambio de división

4.5 Cliente Unity

- ☐ Interfaces `IMatchmakingService` e `IRankedService` con implementación Nakama
- ☐ Medición de `rtt` contra el servidor antes de entrar a la cola
- ☐ Pantalla de selección de modo y tamaño (2, 4, 6)
- ☐ Pantalla de cola con tiempo transcurrido y botón de cancelar
- ☐ Ampliación de ventana de rating: cancelar y reenviar el ticket cada 10 s
- ☐ Carrera rápida: a los 20 s sin rivales suficientes, arrancar con bots
- ☐ Ranked: a los 60 s sin emparejar, ofrecer carrera rápida
- ☐ Al emparejar: unirse al match, leer la sesión, cargar pista y autos
- ☐ El host instancia los bots indicados por la sesión con la IA del sistema de conducción
- ☐ Reincorporación al abrir el juego si `race_session_get` reporta una carrera viva
- ☐ Autodrive en el host para el auto de un jugador desconectado durante la gracia
- ☐ Migración de host: detectar la caída, llamar `race_host_claim`, asumir bots y autodrive
- ☐ Grupos: crear, invitar por código, entrar juntos a la cola
- ☐ Pantalla ranked: división, barra de progreso, puesto, cuenta regresiva de temporada
- ☐ Animación de cambio de rating y de división en resultados
- ☐ Aviso de bloqueo de cola por abandonos

4.6 Pruebas

- ☐ Pruebas unitarias de rating con 2, 4 y 6 humanos, incluyendo empates de rating y abandonos
- ☐ 6 clientes en cola rápida de tamaño 6: una sola carrera con todos
- ☐ 3 clientes en cola rápida de tamaño 4: carrera con 3 humanos y 1 bot al vencer la espera
- ☐ 1 cliente solo: carrera con bots a los 20 s

- ☐ 5 clientes en cola ranked: carrera de 4 humanos y uno sigue en cola
- ☐ Clientes con versión distinta nunca se emparejan
- ☐ Clientes de distinta región nunca se emparejan
- ☐ Grupo de 2 más 2 sueltos: los del grupo quedan en la misma carrera
- ☐ Desconexión de 10 s: el jugador vuelve y termina la carrera
- ☐ Desconexión de 30 s: abandono y último lugar
- ☐ Caída del host a mitad de carrera: la carrera continúa con el sucesor
- ☐ Tres abandonos en ranked: cola bloqueada 15 minutos
- ☐ Con 150 ms de latencia y 5 % de pérdida simulados, la carrera sigue siendo jugable

4.7 Criterio de terminado

- ☐ La cola rápida arranca una carrera en 20 s o menos, con bots si hace falta
- ☐ Una carrera ranked de 4 humanos cambia los ratings según la fórmula y actualiza `ranked_s1`
- ☐ Un jugador que pierde conexión vuelve a su carrera dentro de la gracia
- ☐ La caída del host no termina la carrera
- ☐ Las salas privadas siguen funcionando con el flujo de la Fase 1

Fase 5 — Operación y lanzamiento en pruebas

Objetivo: pasar de un servidor de desarrollo a uno que aguante jugadores reales en América y Europa, con cuentas que siguen al jugador entre móvil y PC. Al terminar esta fase el juego se puede publicar en pruebas.

5.1 Infraestructura

- ☐ Tres entornos separados (local, staging, producción), cada uno con su base de datos y sus claves
- ☐ Claves por defecto cambiadas en staging y producción: server key, HTTP key, clave de firma de sesión, usuario y contraseña de la consola
- ☐ Secretos fuera del repositorio, inyectados por variables de entorno
- ☐ Proxy inverso con TLS delante de Nakama (API y socket)
- ☐ Consola de Nakama accesible solo por VPN o lista de IPs
- ☐ PostgreSQL administrado o con copia automática diaria y retención de 14 días
- ☐ Restauración de una copia probada en staging
- ☐ Pipeline: compila TypeScript, corre pruebas, construye la imagen Docker con versión, despliega a staging

- ☐ Despliegue a producción con un paso manual de aprobación
- ☐ Procedimiento escrito de despliegue y de vuelta atrás
- ☐ Métricas de Nakama exportadas a Prometheus, con panel de sesiones, latencia de RPC, errores, CPU, memoria y conexiones a la base
- ☐ Alertas: servidor caído, tasa de errores, latencia alta, disco de la base
- ☐ Logs centralizados y buscables por ID de sesión de carrera y por jugador
- ☐ Límite de frecuencia activo en todos los RPC

5.2 Regiones: América y Europa

- ☐ Servidor principal (cuentas y metajuego) desplegado en la costa este de Estados Unidos
- ☐ Prueba de concepto de nodo de relay en Europa: segundo Nakama solo para cola y carrera, con el servidor principal como única fuente de cuentas y datos
- ☐ Variable `NODE_ROLE` (`home` o `relay`) que decide qué módulos registra cada nodo
- ☐ RPC `relay_token` en el principal: emite un token firmado de corta duración con el ID del jugador y su región
- ☐ Autenticación en el nodo de relay validando ese token en un hook previo a la autenticación
- ☐ En el nodo de relay, el hook `matchmakerMatched` crea la sesión llamando al principal de servidor a servidor con la HTTP key
- ☐ `config_get` devuelve la lista de regiones con su dirección de relay
- ☐ Decisión registrada tras la prueba: relay regional (recomendado) o dos despliegues independientes por región como alternativa
- ☐ Latencia medida desde Colombia, México, Brasil, España y Alemania hacia cada región

5.3 Cuentas en móvil y PC

- ☐ Móvil: inicio como invitado con ID de dispositivo
- ☐ PC: método de acceso definido según la tienda elegida (cuenta de la tienda o correo)
- ☐ Enlace de Google, Apple y correo a una cuenta existente
- ☐ Inicio de sesión desde otra plataforma con un método enlazado, recuperando todo el progreso
- ☐ Conflicto de enlace: el servidor devuelve el resumen de ambas cuentas para que el jugador elija
- ☐ `account_delete` con confirmación y borrado de datos personales
- ☐ Aviso de enlazar cuenta desde el nivel 5 con premio único al enlazar

5.4 Servidor: LiveOps base, inbox y analítica

- ☐ Objeto `liveops/config` con flags, valores ajustables y calendario vacío, servido por `config_get`
- ☐ Flag de mantenimiento con mensaje y lista de jugadores exentos
- ☐ Versión mínima de cliente por plataforma
- ☐ Módulo `inbox` : correo con premio adjunto y vencimiento, `inbox_list`, `inbox_claim` idempotente
- ☐ RPCs de soporte protegidos con HTTP key: dar y quitar objetos, enviar correo a uno o a todos, marcar y sancionar
- ☐ Eventos de analítica desde el servidor: registro, sesión, carrera completada, movimiento de wallet, compra en tienda, emparejamiento
- ☐ Destino de analítica elegido y conectado
- ☐ RPC de soporte para limpiar sesiones de carrera con más de 7 días

5.5 Cliente Unity

- ☐ Selector de entorno por configuración de build
- ☐ Dos conexiones: principal siempre, relay regional durante cola y carrera
- ☐ Elección automática de región por menor latencia, con cambio manual en ajustes
- ☐ Reconexión automática del socket principal con espera creciente
- ☐ Pantalla de ajustes de cuenta: métodos enlazados, enlazar, cerrar sesión, borrar cuenta
- ☐ Pantalla de conflicto de enlace
- ☐ Pantallas de mantenimiento y de actualización obligatoria
- ☐ Inbox con indicador de no leídos y reclamo de premios
- ☐ Eventos de analítica de cliente: embudo del primer inicio, pantallas, rendimiento
- ☐ Reporte de errores y caídas del cliente
- ☐ Builds de Android, iOS y PC apuntando a producción

5.6 Publicación en pruebas

- ☐ Política de privacidad y términos de uso publicados y enlazados desde el juego
- ☐ Clasificación por edad y formularios de datos de cada tienda completados
- ☐ Canal de pruebas configurado en cada tienda (pruebas cerradas o acceso anticipado)
- ☐ Canal para reportes de los jugadores de prueba

5.7 Pruebas

- ☐ Cliente sin gráficos que inicia sesión, entra a cola, corre y reporta resultado

- ☐ Prueba de carga: 500 jugadores simultáneos en carreras de 6 durante 1 hora, sin errores ni degradación
- ☐ Prueba de resistencia: 100 jugadores durante 12 horas sin crecimiento de memoria
- ☐ Despliegue y vuelta atrás ejecutados siguiendo el procedimiento
- ☐ Carrera entre dos jugadores de Europa por el relay europeo con el metajuego en el principal
- ☐ Cuenta creada en Android, enlazada y abierta en PC con el mismo progreso
- ☐ Borrado de cuenta: los datos desaparecen y el jugador sale de las tablas
- ☐ Modo mantenimiento activado y desactivado sin reiniciar el servidor

5.8 Criterio de terminado

- ☐ La prueba de carga pasa y su resultado queda documentado como techo conocido
- ☐ Se restauró una copia de la base en staging
- ☐ Jugadores de América y de Europa corren con latencia aceptable en su región
- ☐ Un jugador continúa su progreso al cambiar de móvil a PC
- ☐ El juego está disponible para jugadores de prueba en al menos una tienda móvil y en PC

Fase 6 — Misiones, logros y pase

Objetivo: dar motivos para volver cada día y una meta de 6 semanas, y cerrar la primera temporada de ranked con sus premios.

6.1 Diseño y contratos

- ☐ Lista de al menos 20 misiones diarias y 10 semanales con filtros, meta y premio
- ☐ Lista de al menos 20 logros permanentes
- ☐ Tabla del pase: 40 niveles, XP por nivel, premio gratuito y premio premium de cada nivel
- ☐ Cálculo de ritmo: con juego diario normal el nivel 40 se alcanza hacia la semana 5
- ☐ Premios de fin de temporada por división y para el top 100
- ☐ Contrato escrito de `missions_get`, `mission_claim`, `mission_reroll`, `achievements_get`, `achievement_claim`, `pass_get`, `pass_claim`, `pass_buy_premium`

6.2 Servidor: motor de contadores

- ☐ Definición por datos: evento, filtros (modo, pista, clase, posición máxima, tamaño) y meta
- ☐ Evaluador que recibe `RaceCompleted` y devuelve qué contadores avanzan

- ☐ Solo resultados no rechazados avanzan contadores
- ☐ Soporte de contadores de valores distintos (por ejemplo, pistas diferentes)
- ☐ Pruebas unitarias del evaluador con cada tipo de filtro

6.3 Servidor: módulo `missions`

- ☐ Catálogos `missions_daily`, `missions_weekly` y `achievements`
- ☐ Objetos `missions/daily` y `missions/weekly` con sello de fecha, misiones asignadas y progreso
- ☐ Asignación perezosa: 3 misiones por semilla de jugador y fecha UTC al detectar cambio de día o de semana
- ☐ `missions_get` con misiones, progreso y tiempo restante
- ☐ `mission_claim` idempotente: paga monedas y XP de pase
- ☐ `mission_reroll`: un cambio gratuito por día
- ☐ Misiones bloqueadas hasta el nivel 3
- ☐ Logros con el mismo motor, sin vencimiento: `achievements_get`, `achievement_claim`

6.4 Servidor: módulo `pass`

- ☐ Catálogo `pass_s{N}` ligado a las fechas de la temporada
- ☐ Objeto `pass/s{N}`: XP de pase, niveles reclamados por carril, premium sí o no
- ☐ XP de pase por misión reclamada y, en menor medida, por carrera terminada
- ☐ `pass_get` con nivel, progreso y estado de cada premio
- ☐ `pass_claim` por nivel y carril, idempotente
- ☐ `pass_buy_premium` con gemas; habilita reclamar los niveles premium ya alcanzados
- ☐ Función de entrega de pase premium reutilizable para la compra con dinero real de la Fase 9

6.5 Servidor: cierre de temporada

- ☐ Detección perezosa de temporada vencida en el primer acceso del jugador
- ☐ Lectura de división más alta y puesto final en `ranked_s{N}`
- ☐ Premios de división y de top 100 enviados al inbox
- ☐ Premios de pase alcanzados y no reclamados enviados al inbox
- ☐ Creación de `ranked/s{N+1}` con reinicio parcial: $1000 + (\text{rating} - 1000) \times 0,5$
- ☐ Creación de la tabla `ranked_s{N+1}` y del pase de la nueva temporada
- ☐ Marca de cierre procesado por jugador para que ocurra una sola vez

6.6 Cliente Unity

- ☐ Interfaces `IMissionService`, `IAchievementService` e `IPassService` con implementación Nakama
- ☐ Pantalla de misiones con pestañas diaria y semanal, barras de progreso y cuenta regresiva
- ☐ Botón de cambiar misión y botón de reclamar con animación de premio
- ☐ Progreso de misiones mostrado en la pantalla de resultados
- ☐ Pantalla de logros con progreso y reclamo
- ☐ Pantalla de pase: carril gratuito y premium, nivel actual, vista previa de premios
- ☐ Compra de pase premium con gemas
- ☐ Indicadores de premios por reclamar en el menú principal
- ☐ Pantalla de fin de temporada con división final y premios

6.7 Pruebas

- ☐ Cambio de día UTC simulado: misiones nuevas y lo no reclamado se pierde
- ☐ Dos jugadores distintos reciben misiones distintas el mismo día
- ☐ Una misión solo avanza con carreras que cumplen su filtro
- ☐ Reclamar dos veces la misma misión paga una vez
- ☐ Reclamar un nivel de pase no alcanzado: rechazado
- ☐ Comprar premium en el nivel 12 permite reclamar los 12 premios premium
- ☐ Cierre de temporada adelantado en staging: premios en el inbox y temporada 2 abierta con rating reiniciado
- ☐ Cierre procesado dos veces para el mismo jugador: premia una vez

6.8 Criterio de terminado

- ☐ Las misiones cambian solas con el día UTC y solo avanzan con carreras reales
- ☐ Un jugador completa misiones, sube niveles de pase y reclama premios de ambos carriles
- ☐ La temporada 1 cierra y abre la 2 sin intervención manual
- ☐ Añadir una misión nueva requiere solo editar el catálogo

Fase 7 — Social

Objetivo: que los jugadores se encuentren, corran juntos y pertenezcan a un club, con chat moderado desde el primer día.

7.1 Diseño y contratos

- ☐ Reglas de club escritas: 30 miembros, costo de creación, roles, puntos por carrera (3 victoria, 2 podio, 1 terminar)
- ☐ Lista de frases rápidas para sala y carrera
- ☐ Lista de palabras bloqueadas por idioma (español, inglés, portugués)
- ☐ Contrato escrito de `friend_code_get`, `friend_add_by_code`, `recent_rivals_get`, `invite_send`, `club_create`, `club_get`, `club_update`, `club_search`, `report_player`

7.2 Servidor: amigos y presencia

- ☐ Código de amigo corto por jugador y alta por código
- ☐ Búsqueda por nombre visible
- ☐ Suscripción a `RaceCompleted`: guarda los últimos 20 rivales humanos de cada jugador
- ☐ `recent_rivals_get`
- ☐ `invite_send`: notificación en tiempo real a un amigo con el código de grupo o de sala; persistente si está desconectado, con vencimiento
- ☐ Límite de invitaciones por minuto
- ☐ Bloqueo: un jugador bloqueado no puede invitar, escribir ni unirse a la sala del otro
- ☐ Vista de amigos de las leaderboards verificada con la lista real

7.3 Servidor: clubes

- ☐ `club_create`: cobra monedas, valida nombre y lema con el filtro, crea el group con metadata (emblema, lema, región, división mínima)
- ☐ `club_update` solo para líder y administradores; la metadata nunca se escribe desde el cliente
- ☐ `club_search` por nombre, región y cupo disponible
- ☐ `club_get` con miembros, roles, aporte semanal de cada uno y puesto del club
- ☐ Hook `before` de unirse al grupo: requisito de división mínima y nivel 8
- ☐ Un jugador solo puede estar en un club; el club queda guardado en su perfil
- ☐ Tabla `club_week` con el group como dueño
- ☐ Suscripción a `RaceCompleted`: suma puntos al club del jugador y a su aporte semanal
- ☐ Callback de reinicio semanal: premio al club ganador y extra a sus 3 mejores miembros por inbox
- ☐ Vista de club en `lb_get`

7.4 Servidor: chat y moderación

- ☐ Canal de club persistente con historial de 7 días
- ☐ Chat directo solo entre amigos mutuos
- ☐ Hook `before` de envío de mensaje: filtro de palabras, límite de 1 por segundo y 20 por minuto, longitud máxima
- ☐ Rechazo de entrada a canales no permitidos (sin chat global)
- ☐ `report_player` con contexto: sesión, canal, últimos mensajes del reportado
- ☐ Silencio automático temporal con 3 reportes de jugadores distintos en 24 h
- ☐ RPC de soporte para ver reportes, silenciar, levantar silencio y sancionar
- ☐ Jugador silenciado: el hook rechaza sus mensajes y le informa hasta cuándo

7.5 Cliente Unity

- ☐ Interfaces `IFriendService`, `IPartyService`, `IClubService`, `IChatService` con implementación Nakama
- ☐ Lista de amigos con estado: desconectado, en menú, en cola, en carrera, en sala con cupo
- ☐ Publicación del estado propio al cambiar de pantalla
- ☐ Añadir por código, por búsqueda y desde rivales recientes o resultados
- ☐ Solicitudes entrantes, aceptar, rechazar, bloquear
- ☐ Invitar a grupo y a sala privada; aviso de invitación con aceptar y rechazar
- ☐ Grupo visible en el menú, con líder y cola conjunta
- ☐ Buscador y creación de club
- ☐ Pantalla de club: miembros, aportes, puesto semanal, gestión de roles y expulsión
- ☐ Chat de club y chat directo con historial
- ☐ Frases rápidas en sala y en carrera enviadas como código por el relay
- ☐ Botón de reportar y de bloquear en perfil, chat y resultados
- ☐ Ajuste para desactivar el chat de texto

7.6 Pruebas

- ☐ Dos amigos entran juntos a una cola desde una invitación y quedan en la misma carrera
- ☐ Invitación a un jugador desconectado: la ve al conectarse si no venció
- ☐ Jugador bloqueado no puede invitar ni escribir
- ☐ Crear club sin monedas suficientes: rechazado
- ☐ Unirse a un club lleno o sin la división mínima: rechazado
- ☐ Una carrera suma puntos al club y al aporte del miembro

- ☐ Reinicio semanal simulado: premio al club ganador en el inbox
- ☐ Mensaje con palabra bloqueada no se publica
- ☐ Más de 20 mensajes en un minuto: rechazados
- ☐ Tres reportes de jugadores distintos silencian al reportado

7.7 Criterio de terminado

- ☐ Un jugador añade a un rival tras una carrera, lo invita y corren juntos
- ☐ Un club se crea, recibe miembros, suma puntos y aparece en su tabla semanal
- ☐ Todo canal de texto pasa por filtro, límite de frecuencia y reporte
- ☐ Los requisitos de las tiendas para chat (bloquear y reportar visibles) están cubiertos

Fase 8 — Torneos y eventos

Objetivo: ofrecer competencias con premio que abren y cierran solas, un calendario de eventos manejado por configuración, y la telemetría necesaria para que esos premios no se los lleven los tramposos.

8.1 Diseño y contratos

- ☐ Definición de los tres formatos: contrarreloj (mejor tiempo, 20 intentos), copa (puntos en 10 carreras), copa de clubes
- ☐ Tabla de puntos de copa para 2, 4 y 6 autos
- ☐ Premios por tramos de puesto de cada formato
- ☐ Tipos de evento del calendario: XP doble, monedas extra, pista destacada, torneo, oferta especial
- ☐ Umbrales de la detección estadística escritos y revisables
- ☐ Contrato escrito de `tournament_list`, `tournament_join`, `tournament_get`, `events_get`

8.2 Servidor: módulo `tournaments`

- ☐ Catálogo `tournaments` con formato, pista, clase, fechas, costo de entrada, intentos y premios
- ☐ Creación de los torneos de Nakama desde el catálogo al arrancar
- ☐ `tournament_list` con activos y próximos, y el estado del jugador en cada uno
- ☐ `tournament_join`: nivel 10, cobro de entrada, inscripción
- ☐ Contrarreloj: suscripción a `RaceCompleted` que escribe el mejor tiempo y descuenta un intento

- ☐ Copa: modo propio en el matchmaker y puntos por posición a la tabla del torneo
- ☐ Copa de clubes: tabla con el club como dueño
- ☐ `tournament_get` con tabla, puesto del jugador, intentos restantes y premios
- ☐ Callback de fin de torneo: reparto de premios por tramos a través del inbox
- ☐ Retención de 24 h de los premios del top 10 y liberación manual o automática
- ☐ Resultados marcados excluidos de la tabla del torneo

8.3 Servidor: calendario de eventos

- ☐ Lista de eventos en `liveops/config` con tipo, parámetros, inicio y fin
- ☐ Función `activeEvents(ahora)` usada por economía, tienda y matchmaking
- ☐ Multiplicadores de XP y monedas aplicados durante el evento
- ☐ `events_get` para el cliente, con cuenta regresiva
- ☐ Notificación de inbox al iniciar un evento

8.4 Servidor: anti-trampas etapa 1

- ☐ Parciales por checkpoint guardados con cada resultado
- ☐ Tiempo mínimo plausible por tramo entre checkpoints en el catálogo de pistas
- ☐ Detección de parcial imposible en un tramo
- ☐ Detección de mejora brusca frente al historial del jugador
- ☐ Detección de desacuerdo de quórum repetido en un mismo jugador
- ☐ Objeto de marcas por jugador con historial y nivel de sanción
- ☐ Marca oculta: los resultados del jugador dejan de entrar a tablas y torneos
- ☐ Mejor intento de contrarreloj guardado con parciales para revisión
- ☐ RPCs de soporte: listar marcados, ver parciales, confirmar, descartar, sancionar, liberar premios
- ☐ Panel de porcentaje de resultados rechazados y marcados por día

8.5 Cliente Unity

- ☐ Interfaces `ITournamentService` e `IEventService` con implementación Nakama
- ☐ Pantalla de eventos y torneos: activos, próximos, cuenta regresiva
- ☐ Detalle de torneo: reglas, premios, tabla, intentos restantes, botón de inscripción
- ☐ Entrada al contrarreloj de torneo y a la cola de copa desde el detalle
- ☐ Aviso de evento activo en el menú y en la pantalla de resultados
- ☐ Aviso de premios de torneo en el inbox
- ☐ Parciales por checkpoint incluidos en cada envío de resultado

8.6 Pruebas

- ☐ Torneo de 1 hora en staging: abre, recibe intentos, cierra y reparte premios sin intervención
- ☐ Intento 21 en un contrarreloj: rechazado
- ☐ Inscripción sin monedas o sin nivel: rechazada
- ☐ Resultado con parcial imposible: marcado y fuera de la tabla
- ☐ Jugador con marca oculta: juega normal y no aparece en tablas
- ☐ Premio del top 10 retenido y luego liberado
- ☐ Evento de XP doble activado por configuración: se aplica sin build y termina solo

8.7 Criterio de terminado

- ☐ Un torneo de fin de semana corre completo sin intervención manual
- ☐ Ningún resultado marcado cobra premio sin revisión
- ☐ Programar un evento es editar la configuración remota
- ☐ Existe una rutina semanal de revisión de marcados con sus herramientas

Fase 9 — Compras reales y anuncios

Objetivo: vender con dinero real en móvil y PC (paquetes de monedas y gemas, skins y colores, autos, personalizaciones, pase de temporada, paquetes de evento y especiales), con cada compra validada por el servidor y entregada una sola vez.

Si se quiere medir ingresos durante el lanzamiento en pruebas, esta fase puede adelantarse a justo después de la Fase 5: solo depende de las fases 3 y 5.

9.1 Diseño y contratos

- ☐ Tienda de PC elegida y su sistema de compras revisado
- ☐ Lista de productos con precio por tienda: paquetes de gemas, paquetes de monedas, paquete inicial, paquetes de skins, autos, pase premium, paquetes de evento
- ☐ Regla definida: qué se compra directo con dinero y qué solo con gemas
- ☐ Catálogo `iap_products` : ID de producto por tienda, contenido como lista de `Reward` , límite de compras, vigencia
- ☐ Contrato escrito de `iap_products_get` , `iap_validate` , `iap_restore`
- ☐ Revisión de las reglas de cada plataforma sobre moneda comprada en otra plataforma

9.2 Configuración de tiendas

- ☐ Productos creados en App Store Connect y en Google Play Console con los mismos identificadores lógicos
- ☐ Productos creados en la tienda de PC
- ☐ Credenciales de validación de Apple y Google cargadas en la configuración de Nakama
- ☐ Notificaciones de servidor de Apple y de Google apuntando al servidor para reembolsos
- ☐ Cuentas de prueba de compra en cada tienda

9.3 Servidor: compras

- ☐ `iap_products_get` : productos visibles para el jugador según plataforma, eventos y límites
- ☐ `iap_validate` para Apple con la validación nativa de Nakama
- ☐ `iap_validate` para Google con la validación nativa de Nakama
- ☐ Validación de compras de PC contra la API de la tienda elegida (integración propia)
- ☐ Entrega del contenido con el ID de transacción como clave de idempotencia
- ☐ Recibo ya visto: responde éxito sin entregar de nuevo
- ☐ Plataforma de origen registrada en cada entrega de gemas
- ☐ Productos no consumibles (autos, skins, pase): `iap_restore` los reentrega si faltan
- ☐ Límite de compras por producto y por jugador
- ☐ Pase premium comprable con dinero usando la función de entrega de la Fase 6
- ☐ Paquetes de evento ligados al calendario de la Fase 8
- ☐ Reembolso notificado: descuenta lo entregado; saldo negativo marca la cuenta
- ☐ Eventos de analítica: compra iniciada, validada, entregada, fallida, reembolsada, con precio y moneda
- ☐ RPC de soporte para consultar las compras de un jugador y reentregar una compra

9.4 Servidor: anuncios con recompensa

- ☐ Lugares de anuncio definidos en el catálogo con premio y tope diario
- ☐ Punto de entrada para la verificación de servidor a servidor de la red de anuncios, con validación de firma
- ☐ Entrega del premio solo al recibir la verificación, con el ID de la impresión como clave de idempotencia
- ☐ Tope diario por jugador y por lugar

9.5 Cliente Unity

- ☐ Interfaz `IPurchaseService` sobre el servicio de compras existente, con implementación por tienda
- ☐ Tienda de dinero real: productos con precio local traído de la tienda
- ☐ Flujo: comprar en la tienda, enviar recibo a `iap_validate`, confirmar la compra a la tienda solo tras la respuesta del servidor
- ☐ Compras pendientes reenviadas al iniciar el juego
- ☐ Botón de restaurar compras
- ☐ Pantalla de entrega con el contenido recibido
- ☐ Mensajes de compra cancelada, pendiente y fallida
- ☐ Oferta de paquete inicial y de paquetes de evento en el menú
- ☐ Anuncio con recompensa: mostrar, esperar la entrega del servidor, actualizar saldos
- ☐ Compras desactivables por configuración remota y por plataforma

9.6 Anti-trampas etapa 2 (solo si el ranked lo necesita)

- ☐ Handler de match autoritativo delgado que reenvía todos los mensajes sin interpretarlos
- ☐ Interpretación de dos códigos de operación: cruce de checkpoint y meta
- ☐ Orden de llegada y tiempos sellados con el reloj del servidor; resultado con confianza `server`
- ☐ El hook `matchmakerMatched` crea el match autoritativo para ranked y torneos
- ☐ Prueba de carga repetida con el handler activo

9.7 Pruebas

- ☐ Compra de prueba en Apple, Google y PC: contenido entregado una vez
- ☐ El mismo recibo enviado dos veces: una sola entrega
- ☐ Recibo falso o de otra aplicación: rechazado
- ☐ Juego cerrado entre el pago y la validación: la compra se entrega al volver a abrir
- ☐ Restaurar compras en un dispositivo nuevo devuelve autos, skins y pase
- ☐ Reembolso de prueba: gemas descontadas
- ☐ Compra en móvil visible en PC con la misma cuenta
- ☐ Anuncio sin verificación del servidor: sin premio
- ☐ Tope diario de anuncios alcanzado: el botón se desactiva

9.8 Criterio de terminado

- ☐ Cada tipo de producto se puede comprar en móvil y en PC y llega a la misma cuenta
- ☐ Ninguna compra se entrega sin validación del servidor ni más de una vez
- ☐ Los reembolsos se reflejan en la cuenta
- ☐ Los ingresos por producto y plataforma se ven en la analítica